



Cloud Foundations



Chapter 5: Virtual Machines

PR. OUASSIM KARRAKCHOU

UNIVERSITE INTERNATIONALE DE RABAT



Virtual Machines

Approaches to Virtualization



The concept of virtual machines existed long before cloud computing was invented.

Based on your own knowledge, what are the different approaches used to implement VMs?



Virtual Machines

Approaches to Virtualization

The technologies used to implement virtual machines can be divided into three broad categories:

- Software Emulation
- Para-virtualization
- Full Virtualization

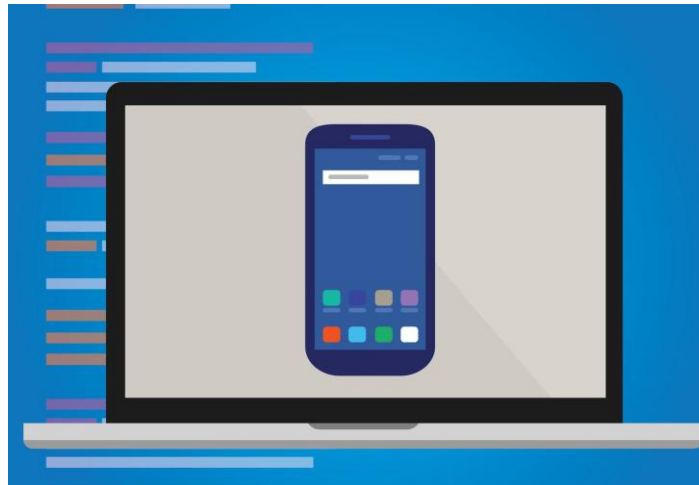
The goal of these 3 approaches is to create a software that **looks and behaves** like a **full computer**; hence the name **Virtual Machine**.





Software Emulation

- **Software emulation** refers to the idea of using **software** to emulate a computer.
- The **software** part is here **very important**: the **code** of the program running on the VM **is treated as data** by the emulator and doesn't run directly on the hardware.
- This idea has many applications. **Can you think of some of them?**

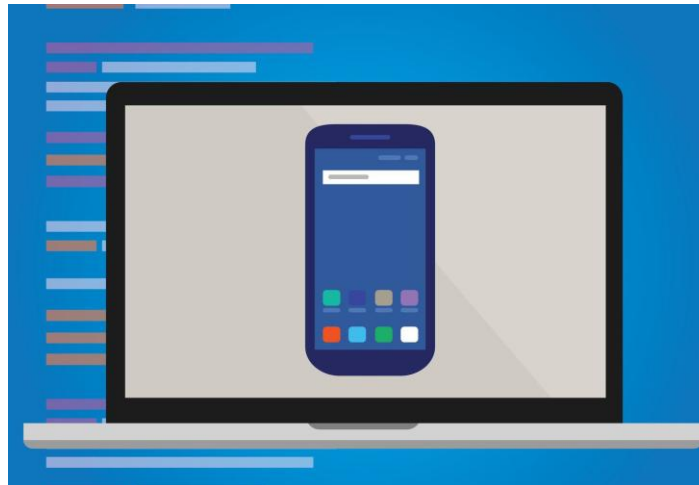




Virtual Machines

Software Emulation Application

- The first application of software emulation is to accommodate heterogeneous computers
- This is especially important if a user is developing a program to be run on another type of computer (e.g., developing Android apps on PC)
- The emulator treats program as data by repeatedly reading an instruction, doing whatever the target computer would do, and then moving to the next instruction.





Virtual Machines

Software Emulation Application

- The software emulation approach has been especially popular as a way to **make a new programming language available quickly**.
- Before a language can be used on a particular type of computer, someone must **write a compiler** that translates programs into instructions the computer understands.
- Writing a compiler is **time-consuming and error prone**, and often limits how quickly a new language can be widely adopted.
- Software emulation allows a language designer to **write one compiler** and then **use the compiler on all types of computers**.

Do you know a programming language that uses this approach?





Virtual Machines

Software Emulation Application

- To implement a **compiler** for a new programming language, use an existing language to build two tools: a **compiler** and a **software emulator**.
- To simplify writing the tools, **define a hypothetical computer** (i.e., a type of virtual machine).
- Build a compiler that **translates programs** written in the new language into **code for the hypothetical machine**.
- Build a **software emulator** that **reads code for the hypothetical machine** and carries out the steps the code specifies.
- The community uses the term **byte code** to describe **instructions** for the **hypothetical computer**.



Virtual Machines

Software Emulation Application

What do you think is the main disadvantage of software emulation?

- Software emulation has a major **disadvantage** that makes it unattractive as the primary form of virtualization used in data centers: software emulation adds **significant overhead** resulting in **slow execution**.
- So, although it **increases the portability** of compilers and other apps, software emulation incurs significant execution overhead, making it **ineligible as a primary form of virtualization in a data center**

⇒ We need to take advantage of the hardware to speed up execution

What is the main challenge facing hardware approaches for virtualization?



Virtual Machines

Para-Virtualization

- When doing **virtualization**, we allow **multiple operating systems** (OS) to run on the **same computer at the same time**.
- However, a big goal of the OS is to **manage the hardware**, configure it, and allow applications to communicate with it.
- As a result, the **virtualization** mechanism **cannot allow** a given operating system to **take over the underlying hardware** completely,
- Instead, **para-virtualization** uses a piece of software known as a **hypervisor** to **control multiple operating systems** and move the processors among them analogous to the way a single operating system moves the processor among a set of running processes.



Virtual Machines

Para-Virtualization

- Unlike software emulation, **para-virtualization** allows software to run at **high speed**.
- That is, para-virtualization **avoids** the inherent **overhead of software emulation** and allows the **processor** to **execute instructions directly** with no extra software involved.
- We say that para-virtualization allows instructions to execute **natively**.
- To prohibit a given operating system from commandeering the hardware, para-virtualization requires the **code for an operating system to be altered** before it can be used.
- One of the most famous para-virtualization hypervisors is called **Xen**:





Virtual Machines

Para-Virtualization

- In para-virtualization, all **privileged instructions** (i.e., instructions used to gain control of the hardware and I/O devices) must be **replaced by calls to hypervisor functions**.
- Thus, when an operating system tries to **perform I/O**, control passes to the **hypervisor**, which can choose whether to **perform** the requested operation on behalf of the operating system or **deny** the request.
- To summarize, **para-virtualization** has the advantage of allowing **high-speed execution** and the disadvantage of **requiring code to be altered** to replace privileged instructions before it can be run.



Virtual Machines

Full Virtualization

- The evolution of para-virtualization is **full virtualization**.
- The goal is here to **avoid the overhead of software emulation** like para-virtualization.
- Additionally, we also want the code of the **operating systems to run unaltered**.

=> How can we achieve that?



Virtual Machines

Properties of Full Virtualization

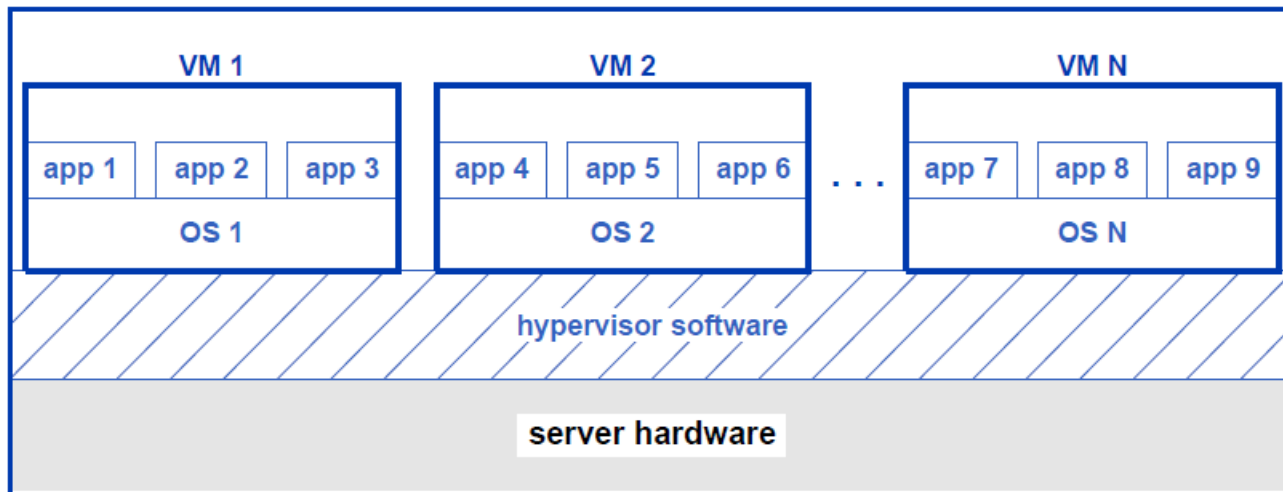
- The full virtualization technologies currently used to support Virtual Machines (VMs) in cloud data centers have three key properties:
 - **Emulation of commercial instruction sets** : To a customer, a VM appears to be identical to a conventional computer, including the complete instruction set. Code that has been compiled to run on a commercial computer will run on a VM unchanged.
 - **Isolated facilities and operation**: Although multiple VMs on a given server can each run an operating system at the same time, the underlying system completely isolates each VM from the others. From the point of view of an operating system running on a VM, the operating system thinks it controls all the machine.
 - **Efficient, low-overhead execution**: Most if not all instructions need to execute natively on the processor without software overhead.



Virtual Machines

Conceptual organization of VM systems

- A full virtualization system looks like this:



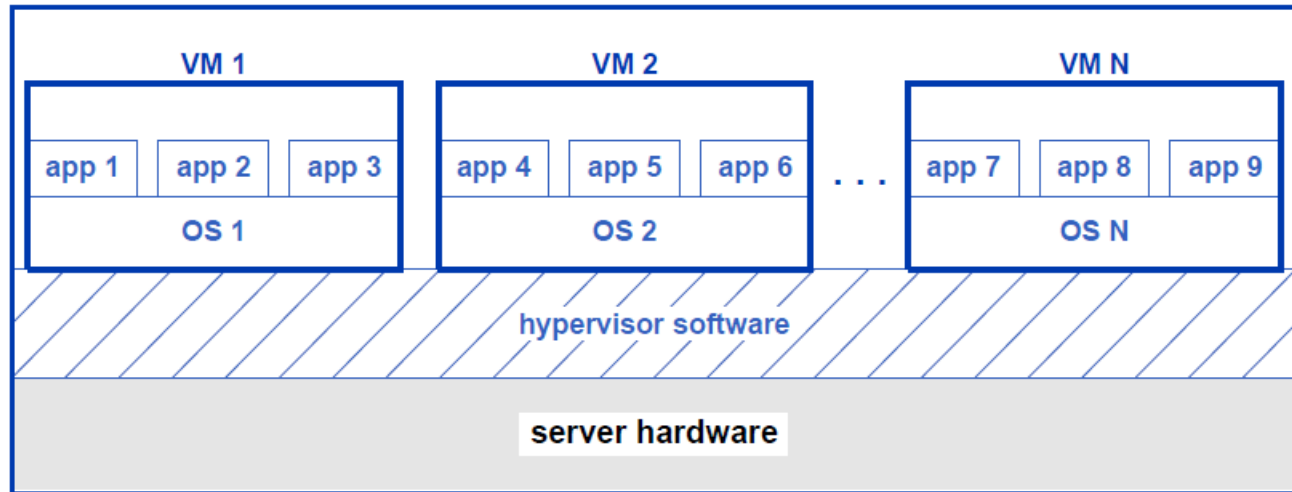
- The key piece of software responsible for creating and managing VMs is known as a **hypervisor**.
- We think of a hypervisor as **controlling the underlying hardware**.
- Each VM the hypervisor creates is **independent** of other VMs.



Virtual Machines

Efficient Execution and Processor Privilege Levels

- In the figure, it appears that **two layers of software** separate a running application from the underlying hardware: an **operating system** and a **hypervisor**.
- We know that **software operates much slower than hardware**, so **how is high speed possible?**

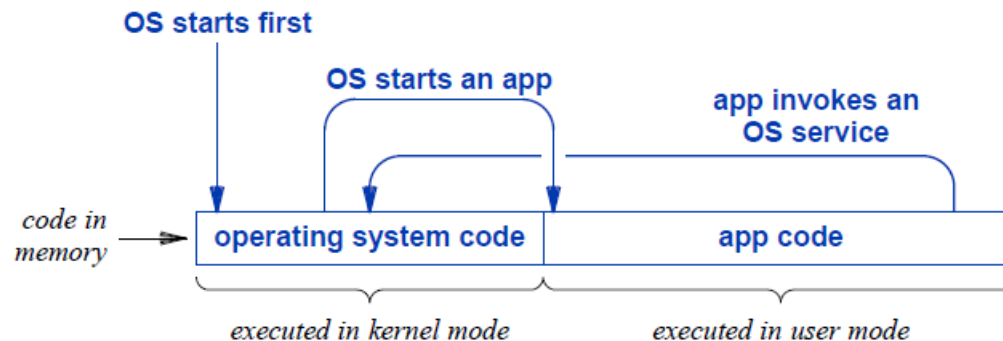




Virtual Machines

Efficient Execution and Processor Privilege Levels

- To understand how a VM runs apps at hardware speed, consider **how an operating system runs apps on a conventional computer**:

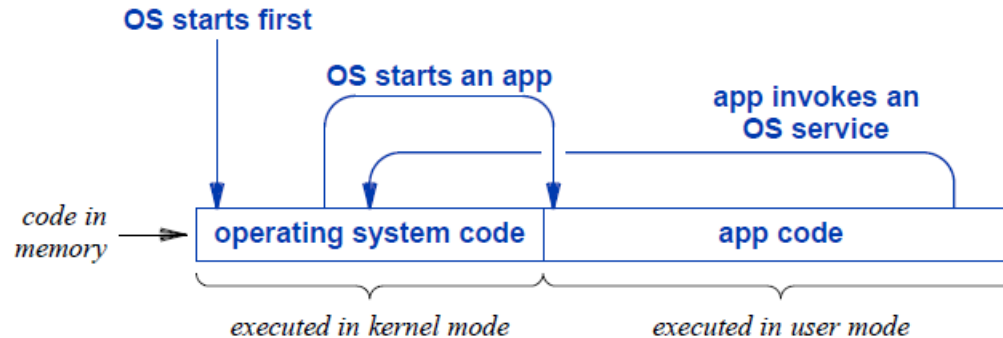


- The main idea here is that **applications** cannot be **allowed to execute all possible instructions**, or the computer would be **vulnerable** to hackers (for example, one app cannot read from the memory of another app).
- To prevent such problems, the **processor** hardware used in a conventional computer has **two privilege levels**.



Virtual Machines

Efficient Execution and Processor Privilege Levels



- Operating system code runs in **kernel mode**, which allows the operating system to perform **all possible instructions**.
- When it switches the **processor to application code**, the operating system also changes to **user mode**, which means only **basic instructions** are available.
- If the application makes a **system call**, to request an operating system service (e.g., to read from a file), the processor **transitions back to kernel mode**.



Virtual Machines

Efficient Execution and Processor Privilege Levels

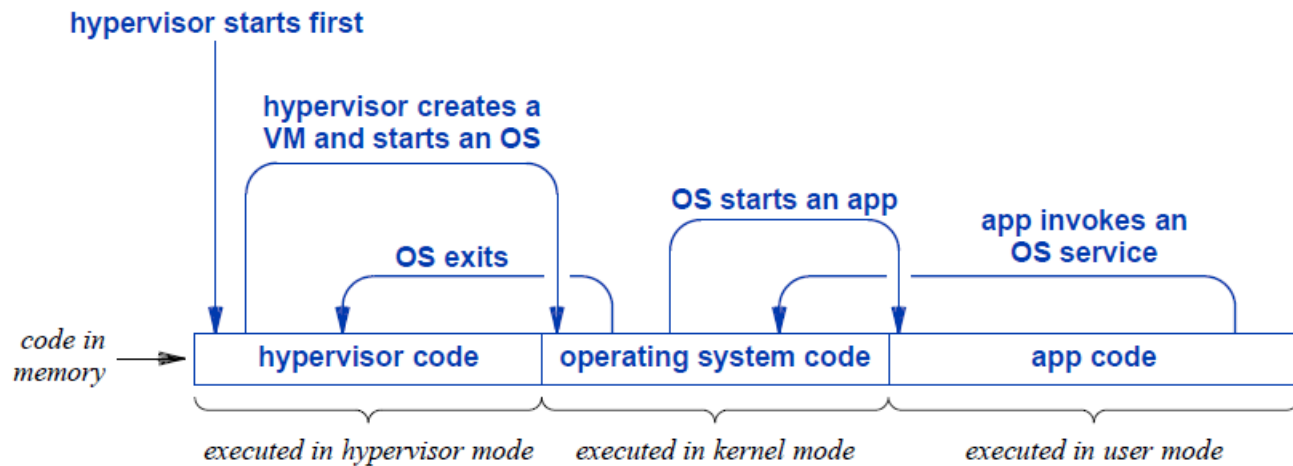
- In **full virtualization**, the idea is to extend the concept of **privilege levels** to the **hypervisor**.
- When a server runs a hypervisor and VMs, the software uses the same approach as an operating system uses to run apps, except that the processor employs **three levels of privilege**: one for the **hypervisor**, a second for an **operating system**, and a third for **apps**.
- Only the **hypervisor** can **create a VM** and allocate **memory** and **processor cores** to the VM.
- The **operating system** is **restricted to the memory** that has been allocated to its VM.
- The **operating system** can **run apps in the memory it has been allocated**. As with a conventional computer, the **processor always runs code directly from memory**, allowing the code to be executed at hardware speed.



Virtual Machines

Efficient Execution and Processor Privilege Levels

- To summarize, in **full virtualization**, the physical server runs code as follows:





Virtual Machines

The problem with I/O devices

- Adding an extra level of privilege makes I/O devices problematic.
- When it boots on a conventional computer, the operating system uses a hardware mechanism known as a bus to communicate with I/O devices.
- The first step consists of sending a series of requests across the bus to form a list of all I/O devices that are present.
- The operating system must include a device driver for each device.
- The operating system uses the device driver code for a given device to control the device hardware and handle all communication with the device.

=> With a hypervisor, we cannot let the OS have complete control on the hardware, otherwise there will be conflicts between the different running OSes



Virtual Machines

Virtual devices

- VM technology uses an interesting approach to handle I/O: **virtual devices**.
- When it creates a VM, a **hypervisor** creates a set of **virtual devices** for the VM.
- A virtual I/O device is **implemented by software**. When an operating system on a VM attempts to use the bus to access an I/O device, the **access violates the privileges of the OS level**, which means the **hypervisor is invoked**.
- The hypervisor runs the appropriate **virtual device software** and then arranges to **send the response** to the operating system as if a **physical device responded**.
- That is, the **hypervisor** makes it appear as if the response **came over the bus**.



Virtual Machines

Virtual devices

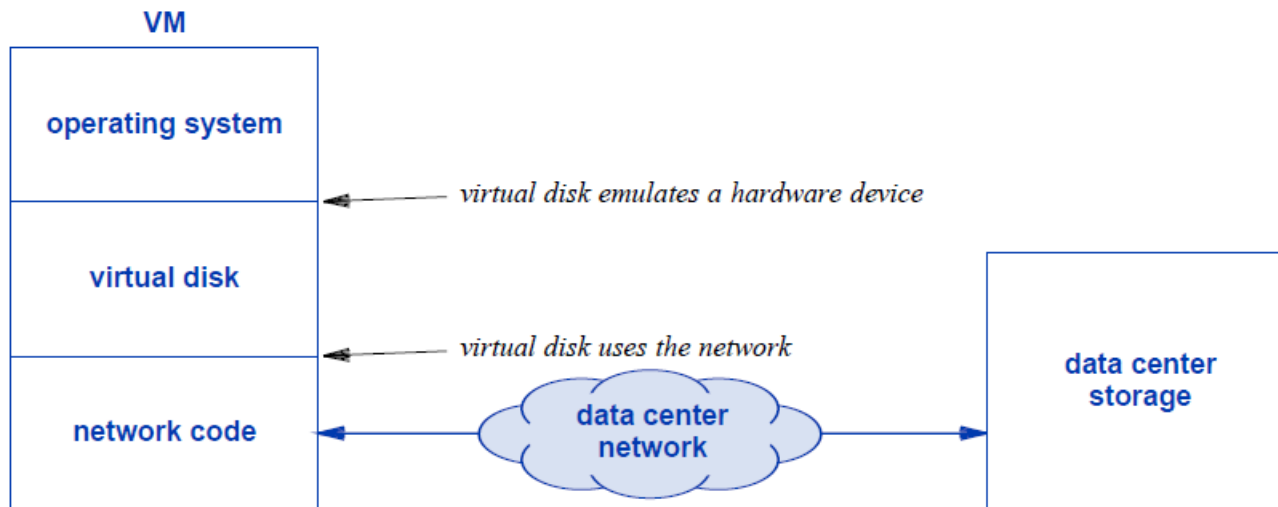
- The use of virtual devices introduces two approaches, and both have been used:
 - **Invent a new, imaginary device** : Most commercial **device hardware** uses a **complicated, arcane interface** that makes writing device driver software difficult. In principle, a programmer who creates a virtual device can create a beautiful, **clean design that avoids the messy hardware details**, making it much easier to write device driver software.
 - **Emulate an existing hardware device** : The alternative to creating an imaginary device consists of building **software that emulates an existing hardware device**. Because the device driver software used with one operating system is incompatible with other operating systems, a **separate driver must be created for each operating system**. Building virtual device software that emulates an existing hardware device **avoids having to build new device drivers**.



Virtual Machines

Virtual devices

- As an example of a virtual device, consider a **virtual disk** that allows communicating with the separate storage over the network in a datacenter:





Virtual Machines

VM as a data object

- A VM differs from a real server in a significant way: a VM is created and managed entirely by software.
- A hypervisor must keep a record of the VM, the region(s) of memory that have been allocated to the VM, the virtual I/O devices that have been created for the VM (including disk space that has been allocated in the data center storage facility), and the current status of the VM (e.g., whether the VM is currently running or has been suspended to allow another VM to run).
- Having a complete record of each VM in a hypervisor has an interesting consequence: a VM can be turned into a digital object. That is, the entire VM can be transformed into a set of bytes.

Can you think of an important usage of this property?



Virtual Machines

VM migration

- The ability to collect all the pieces of a VM into a digital object has great significance, and data centers take advantage of it: **VM migration**.
- Suppose, for example, that a data center manager **receives an alert that a power supply is about to fail** on a server. The manager can **move the VM easily to another server**.
- The basic idea is straightforward: **stop a VM** that is running on one server, **convert the VM to a digital object**, **send the bytes** across the network to a new server, and **resume the VM** on the new server.
- That is, a **hypervisor on one server converts a VM into a digital object**, sends the object to the hypervisor on another server, and the **receiving hypervisor resumes the VM on its server**.
- There are also ways to migrate a VM without stopping it: it's called **live migration**.



Virtual Machines

Running VMs in an application

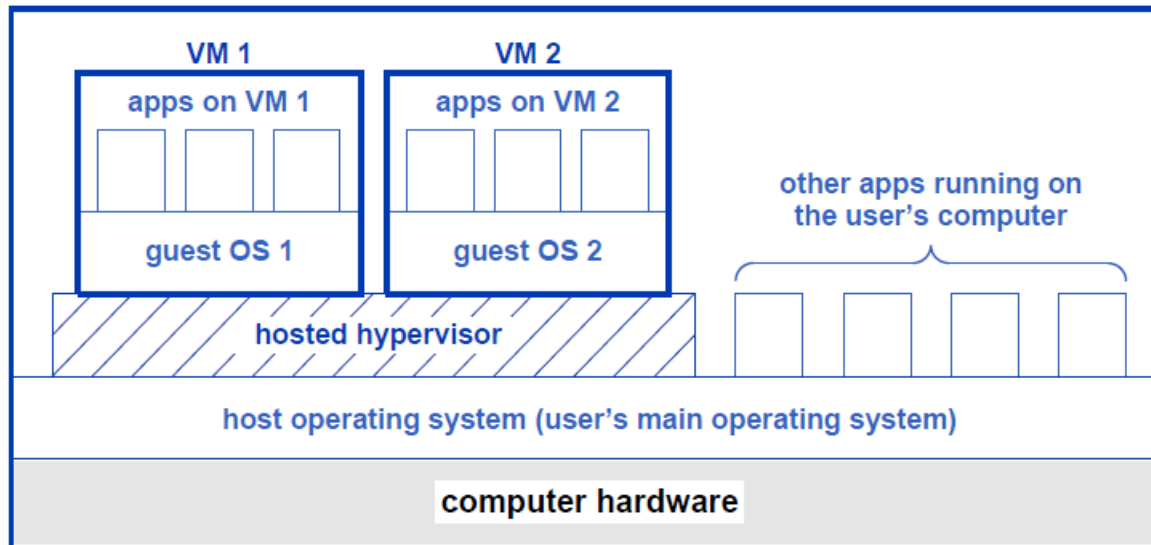
- The previous slides describe how VM technology uses a **hypervisor** that **runs directly on server hardware**.
- Interestingly, an alternative form of VM technology has been developed that allows a **hypervisor to run on a conventional operating system** (i.e., on a user's laptop). A user launches a hypervisor application called a **hosted hypervisor**.
- A **hosted hypervisor runs along with other applications**. Once it starts, the hosted hypervisor allows a user to create and manipulate VMs.
- Two facilities allow a **hosted hypervisor** and a **guest OS** to operate correctly without high privilege: a **processor mechanism that supports virtualization** and a **way to map guest OS operations** onto the host operating system's services.
- These mechanism allow a **guest OS** to proceed as if it were running at the **highest privilege level**, even though it does not have privilege.



Virtual Machines

Running VMs in an application

- The figure below shows the arrangement of software components after a user has launched a hosted hypervisor that runs two VMs along with four other applications:





Virtual Machines

Summary

- *Cloud data centers make extensive use of Virtual Machine (VM) technology. Software known as a hypervisor allows a user to create and manage VMs.*
- *Conceptually, the hypervisor owns the underlying hardware, VMs run over the hypervisor, and each VM runs an operating system plus apps.*
- *A hypervisor provides each operating system with the illusion that it has direct access to peripheral devices over a conventional I/O bus.*
- *All pieces of a VM can be collected into a digital object, which can be sent over a network, allowing hypervisors to migrate a VM from one physical server to another.*
- *An alternative form of VM technology allows a user to run a hosted hypervisor on a conventional computer.*